



# Алгоритмы обработки больших данных

## L04. Stream API, keyed state

Апрель, 2026

Смирнов Сергей

[smirnov.work@mail.ru](mailto:smirnov.work@mail.ru)

[@serg\\_v\\_smirnov](https://twitter.com/serg_v_smirnov)

# Agenda

1. BigData intro
2. Stream & flink intro
3. Batch mode, pyflink runtime architecture
- 4. *DataStream operators API, keyed state***
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
6. State and Chekpoint, State Backends. Table API и SQL API
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

# Homework



Настроить окружение (flink, pg, kafka)

Прочитать данные из таблицы-источника

Записать полученные результаты в файл

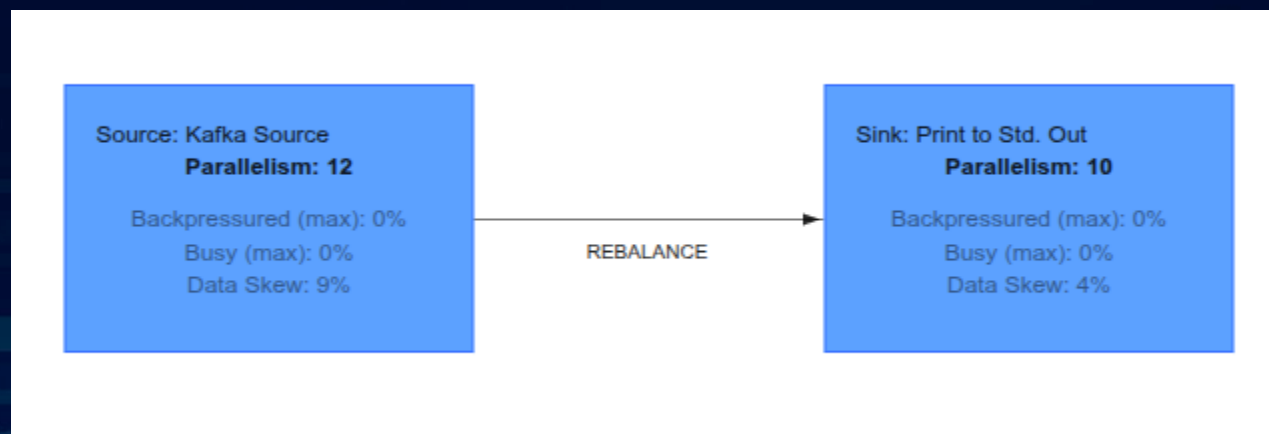
# DataStream API, operators

## read from stream

```
conf = Configuration()
conf.set_integer(key: "rest.port", value: 8081)
env = StreamExecutionEnvironment.get_execution_environment(conf)
env.add_jars("file:///home/serrega/SD/jars/flink-sql-connector-kafka-4.0.1-2.0.jar")
kafka_source = KafkaSource.builder() \
    .set_bootstrap_servers('localhost:9092') \
    .set_topics('test_flink') \
    .set_value_only_deserializer(SimpleStringSchema()) \
    .build()

ds = env.from_source(kafka_source, WatermarkStrategy.no_watermarks(), source_name: "Kafka Source")
ds.print().set_parallelism(10)
```

*filter*  
*flatMap*  
*map*  
*keyby*  
*window*



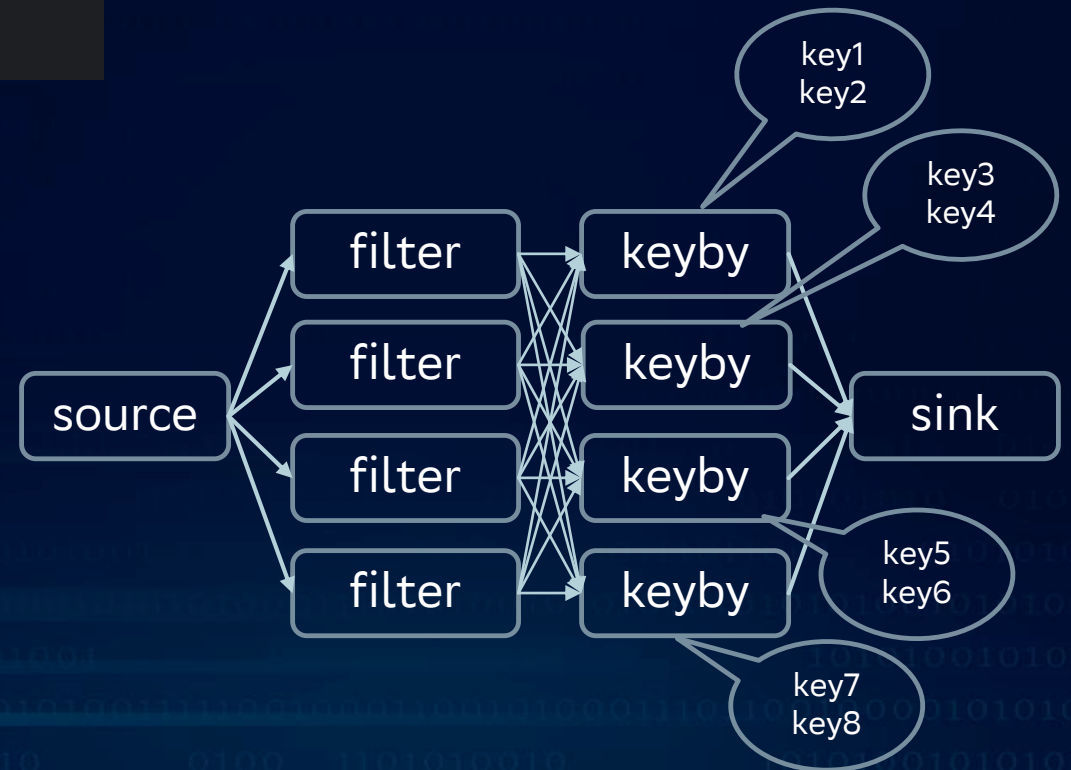
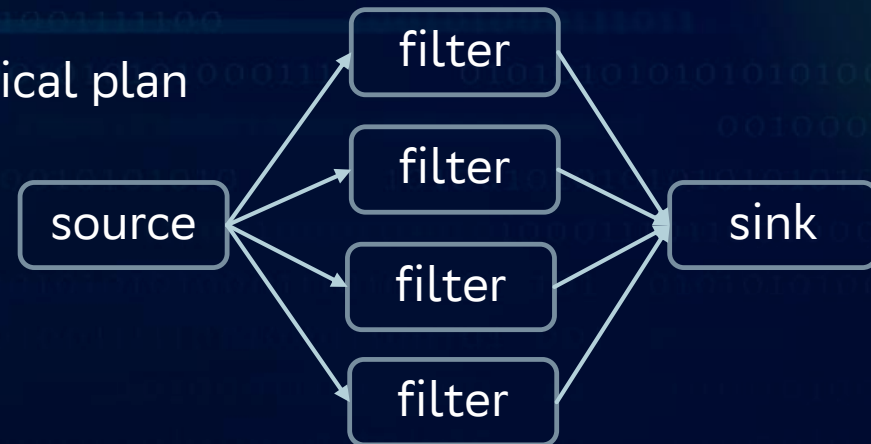
# KeyBy

Logical plan



```
ds2 = ((ds.flat_map(lambda val: [SimpleWord(word, 1) for word in val.split()]  
                        , output_type=Types.PICKLED_BYTE_ARRAY())  
       .key_by(lambda w: w.word))
```

Physical plan





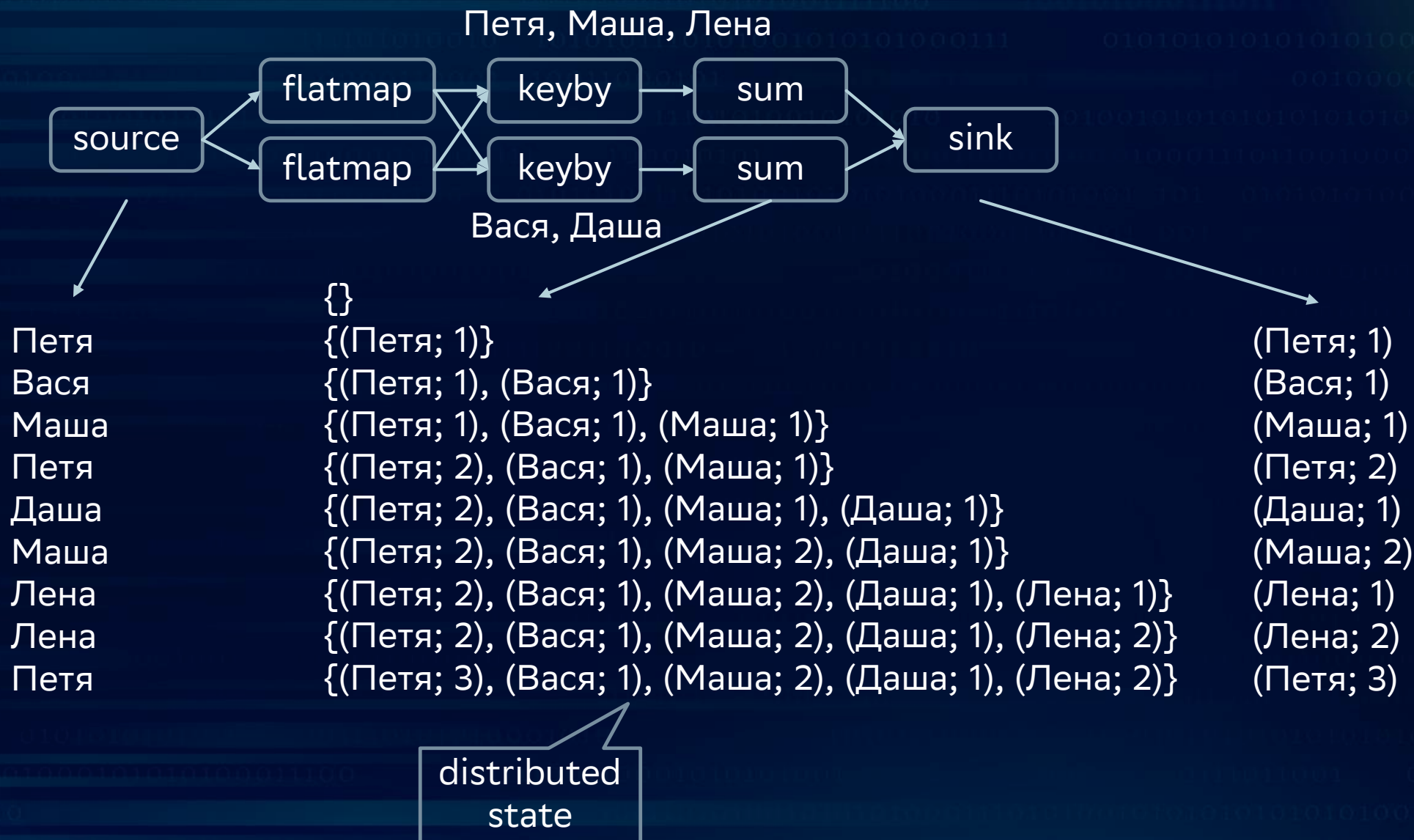
# Sum



## Число проходов через турникет

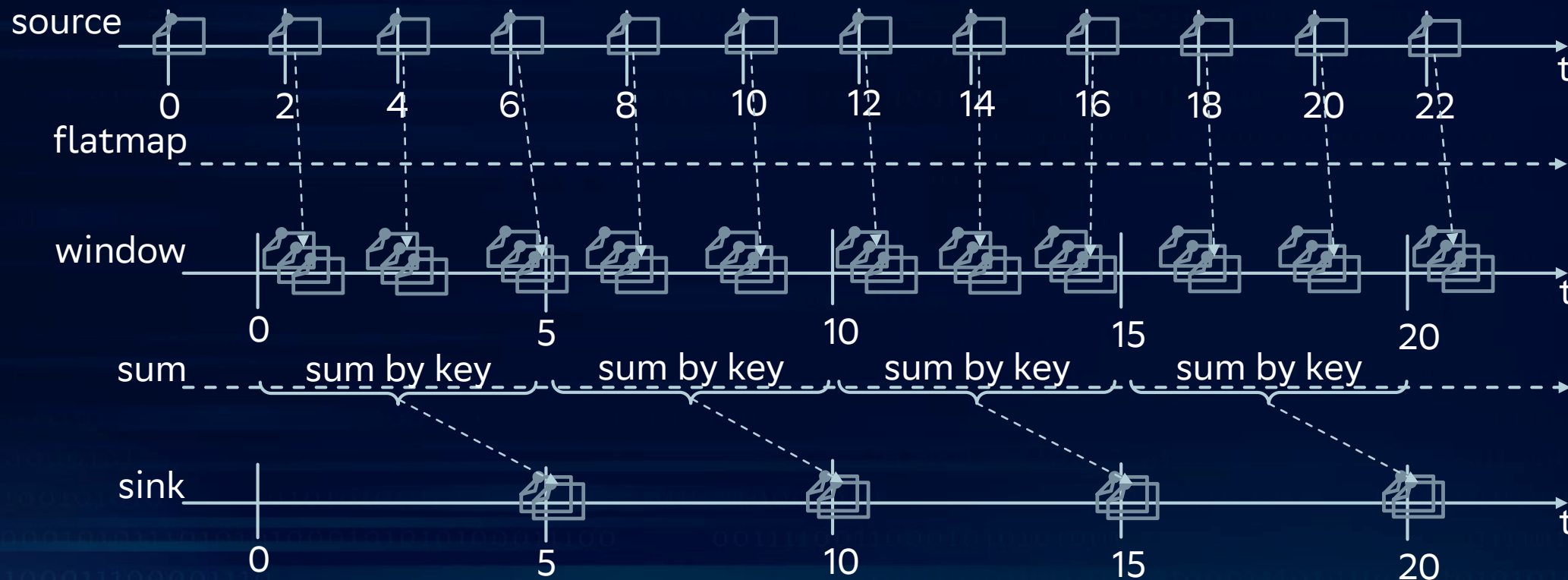


# Sum



# Window, simple example

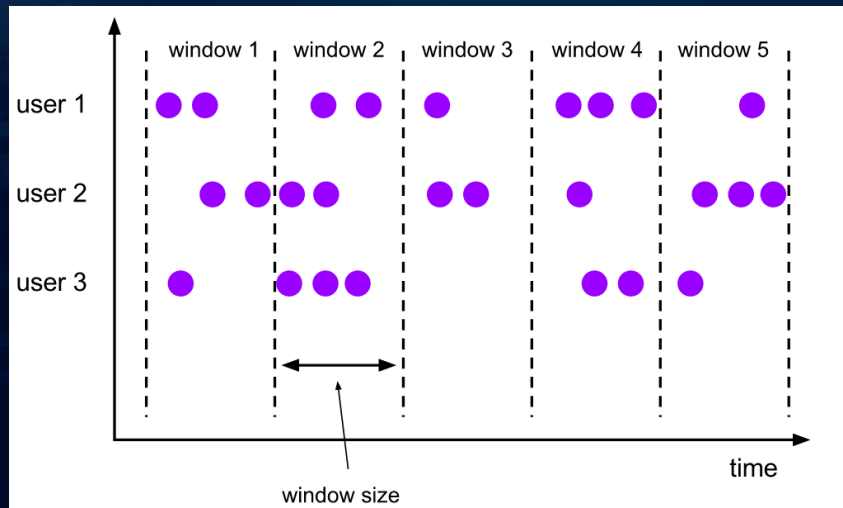
```
window(TumblingProcessingTimeWindows.of(Time.seconds(5)))
```



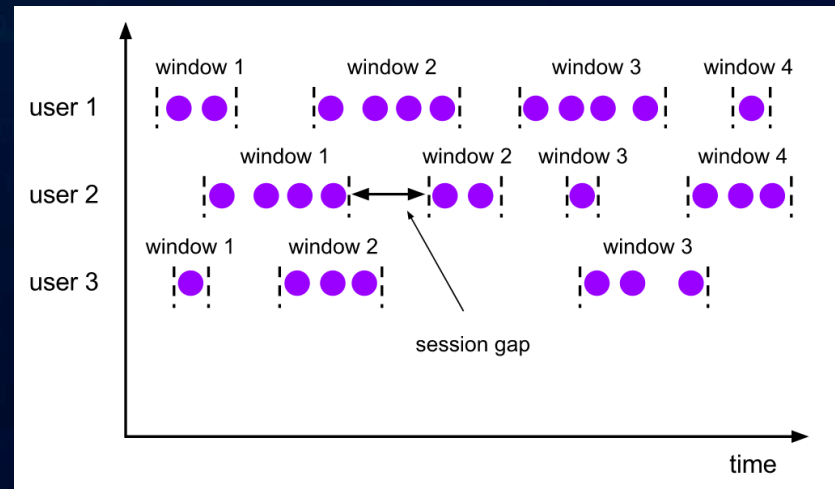


# Windows

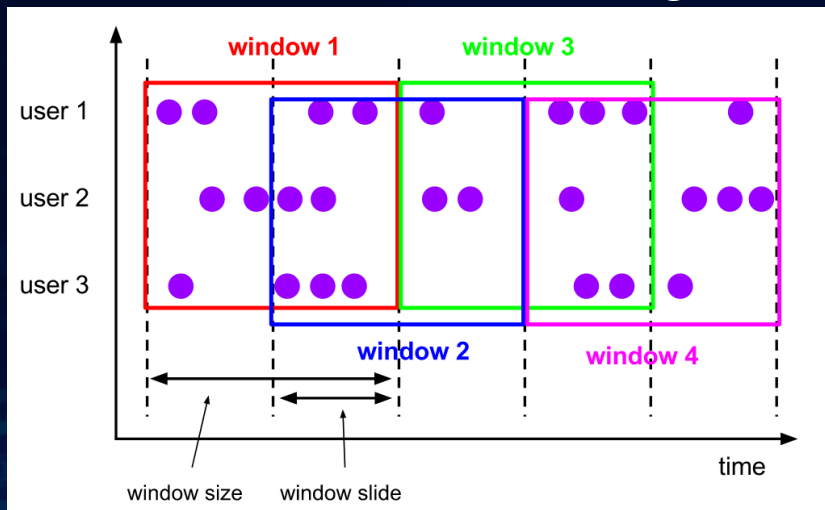
## Tumbling Windows



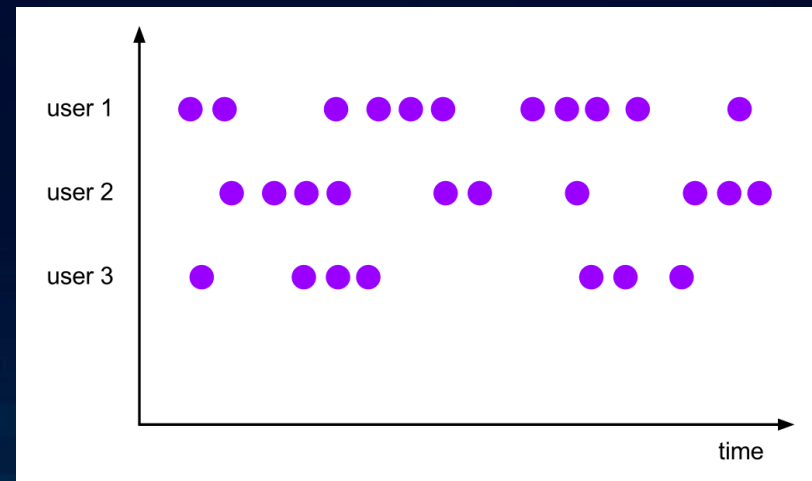
## Session Windows



## Sliding Windows



## Custom Windows



# window vs windowall

|            | <b>window</b>   | <b>windowAll</b>   |
|------------|-----------------|--------------------|
| Stream     | KeyedStream     | DataStream         |
| Operator   | window()        | windowAll()        |
| Processing | В разрезе ключа | По всем сообщениям |
| group by   | keyby()         | -                  |

# Compare with java

```
conf = Configuration()
conf.set_integer("rest.port", 8081)
env = StreamExecutionEnvironment.get_execution_environment(conf)
env.add_jars("file:///home/serega/SD/jars/flink-sql-connector-kafka-4.0.1-2.0.jar")
kafka_source = KafkaSource.builder() \
    .set_bootstrap_servers("localhost:9092") \
    .set_topics("test_flink") \
    .set_value_only_deserializer(SimpleStringSchema()) \
    .build()

ds = env.from_source(kafka_source, WatermarkStrategy.no_watermarks(), "Kafka Source")
ds.flat_map(words_flatmap) \
    .set_parallelism(36) \
    .key_by(word_key) \
    .window(TumblingProcessingTimeWindows.of(Time.seconds(5))) \
    .process(MyCountWindow2(),
        output_type=Types.TUPLE([Types.STRING(), Types.STRING(),
                                   Types.STRING(), Types.INT()]) \
    .print()
```

```
DataStream<String> ds = env.socketTextStream(txtGen, 8888);

ds.flatMap(new FlatMapFunction<String, Tuple2<String, Integer>>() {
    @Override
    public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {
        for(String word: value.split(" ")){
            out.collect(new Tuple2(word, 1));
        }
    }
})
.setParallelism(36)
.keyBy(0)
.timeWindow(Time.seconds(5L))
.sum(1).print();
```

# Homework



- Доработать пример из предыдущего задания:
1. Поменять источник на кафку
  2. Добавить использование оконных функций



# Questions



Спасибо за внимание